JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class Console

java.lang.Object
    java.io.Console

All Implemented Interfaces:

`Flushable`

```
public sealed class Console
extends Object
implements Flushable
```

Methods to access the character-based console device, if any, associated with the current Java virtual machine.

Whether a virtual machine's console exists is dependent upon the underlying platform and also upon the manner in which the virtual machine is invoked. If the virtual machine is started from an interactive command line without redirecting the standard input and output streams, then its console will generally exist and will be connected to the keyboard and display from which the virtual machine was launched. If the standard input or standard output have been redirected (for example, to a file or to a pipe), or if the virtual machine was started from a background job scheduler, the console will not exist.

If the console exists, then it is represented by a unique instance of this class which can be obtained by invoking the `System.console()` method. If the console does not exist, that method will return `null`.

Read and write operations are synchronized to guarantee the atomic completion of critical operations; therefore invoking methods `readLine()`, `readPassword()`, `format()`, `printf()` as well as the read, format and write operations on the objects returned by `reader()` and `writer()` may block in multithreaded scenarios.

Read and write operations use the `Charsets` specified by `stdin.encoding` and `stdout.encoding`, respectively. The `Charset` used for write operations can also be retrieved using the `charset()` method. Since `Console` is intended for interactive use on a terminal, these charsets are typically the same.

Operations that format strings are locale sensitive, using either the specified `Locale`, or the `default format Locale` to produce localized formatted strings.

Invoking `close()` on the objects returned by the `reader()` and the `writer()` will not close the underlying stream of those objects.

The console-read methods return `null` when the end of the console input stream is reached, for example by typing control-D on Unix or control-Z on

Windows. Subsequent read operations will succeed if additional characters are later entered on the console's input device.

Unless otherwise specified, passing a `null` argument to any method in this class will cause a `NullPointerException` to be thrown.

Security note: If an application needs to read a password or other secure data, it should use `readPassword()` or `readPassword(String, Object...)` and manually zero the returned character array after processing to minimize the lifetime of sensitive data in memory.

```
Console cons;
char[] passwd;
if ((cons = System.console()) != null &&
    (passwd = cons.readPassword("[%s]", "Password:")) != null) {
    ...
    java.util.Arrays.fill(passwd, ' ');
}
```

Since:

1.6

Method Summary

All Methods	Instance Methods	

Modifier and Type	Method	Description
`Charset`	**`charset`**`()`	Returns the `Charset` object used for the write operations on this `Console`.
`void`	**`flush`**`()`	Flushes the console and forces any buffered output to be written immediately.
`Console`	**`format`**`(String format, Object... args)`	Writes a formatted string to this console's output stream using the specified format string and arguments with the `default format locale`.
`Console`		

	format(Locale locale, String format, Object... args)	Writes a formatted string to this console's output stream using the specified format string and arguments with the specified locale.
boolean	**isTerminal**()	Returns true if the Console instance is a terminal.
Console	**printf**(String format, Object... args)	A convenience method to write a formatted string to this console's output stream using the specified format string and arguments with the default format locale.
Console	**printf**(Locale locale, String format, Object... args)	A convenience method to write a formatted string to this console's output stream using the specified format string and arguments with the specified locale.
Reader	**reader**()	Retrieves the unique Reader object associated with this console.
String	**readLine**()	Reads a single line of text from the console.
String	**readLine**(String format, Object... args)	Provides a formatted prompt using the default format locale, then reads a single line of text from the console.
String	**readLine**(Locale locale, String format, Object... args)	Provides a formatted prompt using the specified locale, then reads a single line of text from the console.

char[]	**readPassword**()	Reads a password or passphrase from the console with echoing disabled.
char[]	**readPassword**(String format, Object... args)	Provides a formatted prompt using the default format locale, then reads a password or passphrase from the console with echoing disabled.
char[]	**readPassword**(Locale locale, String format, Object... args)	Provides a formatted prompt using the specified locale, then reads a password or passphrase from the console with echoing disabled.
PrintWriter	**writer**()	Retrieves the unique PrintWriter object associated with this console.

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Method Details

writer

```
public PrintWriter writer()
```

Retrieves the unique PrintWriter object associated with this console.

Returns:

The printwriter associated with this console

reader

```
public Reader reader()
```

Retrieves the unique `Reader` object associated with this console.

This method is intended to be used by sophisticated applications, for example, a `Scanner` object which utilizes the rich parsing/scanning functionality provided by the `Scanner`:

```
Console con = System.console();
if (con != null) {
    Scanner sc = new Scanner(con.reader());
    ...
}
```

For simple applications requiring only line-oriented reading, use `readLine(String, Object...)`.

The bulk read operations `read(char[])`, `read(char[], int, int)` and `read(java.nio.CharBuffer)` on the returned object will not read in characters beyond the line bound for each invocation, even if the destination buffer has space for more characters. The `Reader`'s `read` methods may block if a line bound has not been entered or reached on the console's input device. A line bound is considered to be any one of a line feed (`'\n'`), a carriage return (`'\r'`), a carriage return followed immediately by a linefeed, or an end of stream.

Returns:

The reader associated with this console

format

```
public Console format(String format,
                      Object... args)
```

Writes a formatted string to this console's output stream using the specified format string and arguments with the `default format locale`.

Parameters:

`format` - A format string as described in `Format string syntax`.

`args` - Arguments referenced by the format specifiers in the format string. If there are more arguments than format specifiers, the extra arguments are ignored. The number of arguments is variable and may be zero. The maximum number of arguments is limited by the maximum dimension of a Java array as defined by *The Java Virtual Machine Specification*. The behavior on a `null` argument depends on the `conversion`.

Returns:

This console

Throws:

`IllegalFormatException` - If a format string contains an illegal syntax, a format specifier that is incompatible with the given arguments, insufficient arguments given the format string, or other illegal conditions. For specification of all possible formatting errors, see the `Details` section of the formatter class specification.

format

```
public Console format(Locale locale,
                      String format,
                      Object... args)
```

Writes a formatted string to this console's output stream using the specified format string and arguments with the specified `locale`.

Parameters:

`locale` - The locale to apply during formatting. If `locale` is `null` then no localization is applied.

`format` - A format string as described in `Format string syntax`.

`args` - Arguments referenced by the format specifiers in the format string. If there are more arguments than format specifiers, the extra arguments are ignored. The number of arguments is variable and may be zero. The maximum number of arguments is limited by the maximum dimension of a Java array as defined by *The Java Virtual Machine Specification*. The behavior on a `null` argument depends on the `conversion`.

Returns:

This console

Throws:

`IllegalFormatException` - If a format string contains an illegal syntax, a format specifier that is incompatible with the given arguments, insufficient arguments given the format string, or other illegal conditions. For specification of all possible formatting errors, see the `Details` section of the formatter class specification.

Since:

23

printf

```
public Console printf(String format,
                      Object... args)
```

A convenience method to write a formatted string to this console's output stream using the specified format string and arguments with the `default format locale`.

Implementation Requirements:

This is the same as calling `format(format, args)`.

Parameters:

`format` - A format string as described in `Format string syntax`.

`args` - Arguments referenced by the format specifiers in the format string. If there are more arguments than format specifiers, the extra arguments are ignored. The number of arguments is variable and may be zero. The maximum number of arguments is limited by the maximum dimension of a Java array as defined by *The Java Virtual Machine Specification*. The behavior on a `null` argument depends on the `conversion`.

Returns:

This console

Throws:

`IllegalFormatException` - If a format string contains an illegal syntax, a format specifier that is incompatible with the given arguments, insufficient arguments given the format string, or other illegal conditions. For specification of all possible formatting errors, see the `Details` section of the formatter class specification.

printf

```
public Console printf(Locale locale,
                      String format,
                      Object... args)
```

A convenience method to write a formatted string to this console's output stream using the specified format string and arguments with the specified `locale`.

Implementation Requirements:

This is the same as calling `format(locale, format, args)`.

Parameters:

`locale` - The locale to apply during formatting. If `locale` is `null` then no localization is applied.

`format` - A format string as described in `Format string syntax`.

`args` - Arguments referenced by the format specifiers in the format string. If there are more arguments than format specifiers, the extra arguments are ignored. The number of arguments is variable and may be zero. The maximum number of arguments is limited by the maximum dimension of a

Java array as defined by *The Java Virtual Machine Specification*. The behavior on a `null` argument depends on the `conversion`.

Returns:

This console

Throws:

`IllegalFormatException` - If a format string contains an illegal syntax, a format specifier that is incompatible with the given arguments, insufficient arguments given the format string, or other illegal conditions. For specification of all possible formatting errors, see the `Details` section of the formatter class specification.

Since:

23

readLine

```
public String readLine(String format,
                       Object... args)
```

Provides a formatted prompt using the `default format locale`, then reads a single line of text from the console.

Parameters:

`format` - A format string as described in `Format string syntax`.

`args` - Arguments referenced by the format specifiers in the format string. If there are more arguments than format specifiers, the extra arguments are ignored. The number of arguments is variable and may be zero. The maximum number of arguments is limited by the maximum dimension of a Java array as defined by *The Java Virtual Machine Specification*. The behavior on a `null` argument depends on the `conversion`.

Returns:

A string containing the line read from the console, not including any line-termination characters, or `null` if an end of stream has been reached.

Throws:

`IllegalFormatException` - If a format string contains an illegal syntax, a format specifier that is incompatible with the given arguments, insufficient arguments given the format string, or other illegal conditions. For specification of all possible formatting errors, see the `Details` section of the formatter class specification.

`IOError` - If an I/O error occurs.

readLine

```
public String readLine(Locale locale,
                       String format,
                       Object... args)
```

Provides a formatted prompt using the specified `locale`, then reads a single line of text from the console.

Parameters:

`locale` - The locale to apply during formatting. If `locale` is `null` then no localization is applied.

`format` - A format string as described in `Format string syntax`.

`args` - Arguments referenced by the format specifiers in the format string. If there are more arguments than format specifiers, the extra arguments are ignored. The number of arguments is variable and may be zero. The maximum number of arguments is limited by the maximum dimension of a Java array as defined by *The Java Virtual Machine Specification*. The behavior on a `null` argument depends on the `conversion`.

Returns:

A string containing the line read from the console, not including any line-termination characters, or `null` if an end of stream has been reached.

Throws:

`IllegalFormatException` - If a format string contains an illegal syntax, a format specifier that is incompatible with the given arguments, insufficient arguments given the format string, or other illegal conditions. For specification of all possible formatting errors, see the `Details` section of the formatter class specification.

`IOError` - If an I/O error occurs.

Since:

23

readLine

```
public String readLine()
```

Reads a single line of text from the console.

Returns:

A string containing the line read from the console, not including any line-termination characters, or `null` if an end of stream has been reached.

Throws:

`IOError` - If an I/O error occurs.

readPassword

```
public char[] readPassword(String format,
                           Object... args)
```

Provides a formatted prompt using the `default format locale`, then reads a password or passphrase from the console with echoing disabled.

Parameters:

`format` - A format string as described in `Format string syntax` for the prompt text.

`args` - Arguments referenced by the format specifiers in the format string. If there are more arguments than format specifiers, the extra arguments are ignored. The number of arguments is variable and may be zero. The maximum number of arguments is limited by the maximum dimension of a Java array as defined by *The Java Virtual Machine Specification*. The behavior on a `null` argument depends on the `conversion`.

Returns:

A character array containing the password or passphrase read from the console, not including any line-termination characters, or `null` if an end of stream has been reached.

Throws:

`IllegalFormatException` - If a format string contains an illegal syntax, a format specifier that is incompatible with the given arguments, insufficient arguments given the format string, or other illegal conditions. For specification of all possible formatting errors, see the `Details` section of the formatter class specification.

`IOError` - If an I/O error occurs.

readPassword

```
public char[] readPassword(Locale locale,
                           String format,
                           Object... args)
```

Provides a formatted prompt using the specified `locale`, then reads a password or passphrase from the console with echoing disabled.

Parameters:

`locale` - The locale to apply during formatting. If `locale` is `null` then no localization is applied.

`format` - A format string as described in `Format string syntax` for the prompt text.

`args` - Arguments referenced by the format specifiers in the format string. If there are more arguments than format specifiers, the extra arguments are ignored. The number of arguments is variable and may be zero. The maximum number of arguments is limited by the maximum dimension of a Java array as defined by *The Java Virtual Machine Specification*. The behavior on a `null` argument depends on the `conversion`.

Returns:

A character array containing the password or passphrase read from the console, not including any line-termination characters, or `null` if an end of stream has been reached.

Throws:

`IllegalFormatException` - If a format string contains an illegal syntax, a format specifier that is incompatible with the given arguments, insufficient arguments given the format string, or other illegal conditions. For specification of all possible formatting errors, see the `Details` section of the formatter class specification.

`IOError` - If an I/O error occurs.

Since:

23

readPassword

```
public char[] readPassword()
```

Reads a password or passphrase from the console with echoing disabled.

Returns:

A character array containing the password or passphrase read from the console, not including any line-termination characters, or `null` if an end of stream has been reached.

Throws:

`IOError` - If an I/O error occurs.

flush

```
public void flush()
```

Flushes the console and forces any buffered output to be written immediately.

Specified by:

`flush` in interface `Flushable`

charset

```
public Charset charset()
```

Returns the `Charset` object used for the write operations on this `Console`.

The returned charset is used for encoding the data that is sent to the output (e.g., display), specified by the host environment or user. It defaults to the one based on `stdout.encoding`, and may not necessarily be the same as the default charset returned from `Charset.defaultCharset()`.

Returns:

the `Charset` object used for the write operations on this `Console`

Since:

17

isTerminal

```
public boolean isTerminal()
```

Returns `true` if the `Console` instance is a terminal.

This method always returns `true`, since `System.console()` provides a `Console` instance only when both standard input and output are unredirected, that is, when running in an interactive terminal.

Returns:

`true` if the `Console` instance is a terminal

Since:

22

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.